ALNS Complete Detailed Analysis Plan (Full Version)

```
You have:

- 15 fixed instances
- CP-SAT baseline (1 value per instance)
- ALNS-A with 3 seeds per instance
- ALNS-B with 3 seeds per instance
- Objective always defined (no feasibility stage)

=====================================================================
PART I — DATA PREPARATION
=====================================================================

Task 1 — Create Master Data Table

Create one unified dataset with the following fields:

1. Instance ID (1–15)
2. Algorithm (CP-SAT, ALNS-A, ALNS-B)
3. Seed (NA for CP-SAT; 1–3 for ALNS variants)
4. Final objective value f_{i,a,s}
5. Runtime (if fixed limit still record actual runtime)
6. Reference to convergence log (if available)

Deliverable:
One validated master dataframe containing all runs.

---------------------------------------------------------------------

Task 2 — Normalize Objectives

For minimization problems compute relative gap to CP baseline:

gap_{i,a,s} = (f_{i,a,s} - f^{CP}_i) / |f^{CP}_i|

Add columns:
- raw objective
- relative gap to CP

Deliverable:
Extended dataset including normalized performance metrics.

=====================================================================
PART II — PER-INSTANCE AGGREGATION
=====================================================================

Task 3 — Aggregate Seeds Per Instance

For each instance i and algorithm a ∈ {A, B}:

Compute:

1. Mean objective:
   \bar f_{i,a} = (1/3) * Σ_s f_{i,a,s}

2. Median objective

3. Best objective:
   min_s f_{i,a,s}

4. Standard deviation across seeds

5. Minimum and maximum values

6. Mean relative gap

7. Best relative gap

Deliverable:
15-row table per algorithm with full statistics.
---------------------------------------------------------------------
```

Task 4 — Win/Loss Analysis

For each instance i compute comparisons:

1. Mean(ALNS-A) vs Mean(ALNS-B)
2. Best(ALNS-A) vs Best(ALNS-B)
3. Mean(ALNS-A) vs CP-SAT
4. Mean(ALNS-B) vs CP-SAT

Count:
- Wins
- Ties
- Losses

Deliverable:
Win/Tie/Loss summary table.

PART III — STATISTICAL TESTING

Task 5 — Paired Statistical Test (A vs B)

Using per-instance means:

$$d_i = \bar f_{i,A} - \bar f_{i,B}$$

Apply Wilcoxon signed-rank test to $\{d_1, \dots, d_{15}\}$.

Report:
- p-value
- Mean difference
- Median difference

Task 6 — Effect Size

Compute:

1. Mean percentage improvement
2. Median percentage improvement

Explicitly report magnitude independent of statistical significance.

PART IV — STABILITY ANALYSIS

Task 7 — Within-Instance Variability

For each instance and algorithm:

1. Standard deviation across seeds
2. Coefficient of variation:
   $CV_{i,a} = std_{i,a} / \bar f_{i,a}$

Task 8 — Across-Instance Stability

Aggregate:

1. Mean(std_A) vs Mean(std_B)
2. Median(std_A) vs Median(std_B)

Optional: paired comparison across instances.

PART V — CONVERGENCE ANALYSIS

Task 9 — Extract Best-So-Far Curves

For each run record:

$f_{i,a,s}(t)$ = best objective found up to time t.

Task 10 — Aggregate Across Seeds

For each instance and algorithm:

$\bar f_{i,a}(t) = (1/3) * \Sigma_s f_{i,a,s}(t)$

Task 11 — Aggregate Across Instances

For each algorithm:

$F_a(t) = (1/15) * \Sigma_i \bar f_{i,a}(t)$

Plot:
- time vs objective
- time vs relative gap

Task 12 — Time-to-Target Analysis

Define target gap (e.g., 2% above CP).

For each run compute:

$T_{i,a,s}$ = first time target reached.

Aggregate:
- Mean per instance
- Paired comparison A vs B

Optional: cactus plot of time-to-target.

PART VI — OPERATOR USAGE ANALYSIS

Task 13 — Extract Operator Event Log

For each operator application record:

1. Instance
2. Algorithm
3. Seed
4. Operator name
5. Objective before
6. Objective after
7. Δ = objective_after − objective_before
8. Acceptance flag

Task 14 — Per-Run Operator Metrics

For each operator o in run (i,a,s):

1. Selection count
2. Selection share
3. Improvement count ($\Delta < 0$)
4. Improvement probability:
   P(improve | o) = (# improving) / (# selected)
5. Acceptance rate
6. Mean Δ
7. Mean improving Δ

Task 15 — Per-Instance Aggregation

Aggregate operator metrics across 3 seeds per instance.

Task 16 — Global Operator Effectiveness

Across 15 instances compute for each operator:

1. Mean selection share
2. Mean improvement probability
3. Mean improving Δ
4. Standard deviation across instances

Task 17 — Compare Operator Behavior (A vs B)

Compare:

1. Selection distributions
2. Entropy of selection:
   $H = -\sum_o p_o \log(p_o)$
3. Improvement probabilities
4. Acceptance dynamics

Task 18 — Contribution to Final Solutions

For each run identify which operator produced the final best improvement.

Compute frequency per operator across all runs.

===
PART VII — OPTIONAL ADVANCED ANALYSIS
===

Task 19 — Early vs Late Phase Analysis

Split runtime into:

- First 30%
- Middle 40%
- Last 30%

Compute operator effectiveness per phase.

Task 20 — Improvement per Time Unit

If operator runtime known compute:

Efficiency_o = (Total improvement by o) / (Total time spent in o)

===
PART VIII — VISUALIZATION REQUIREMENTS
===

1. Boxplots of final relative gaps.
2. Scatter plot per instance (Mean A vs Mean B).
3. Global convergence curves.
4. Stability comparison plots.
5. Operator selection heatmap.
6. Operator improvement probability charts.
7. Optional cactus plot.

===
MINIMAL DEFENSIBLE CORE
===

If time constrained:

1. Per-instance mean aggregation.
2. Wilcoxon A vs B.
3. Stability comparison.
4. Operator improvement probability + selection share.

This constitutes a statistically coherent evaluation protocol.